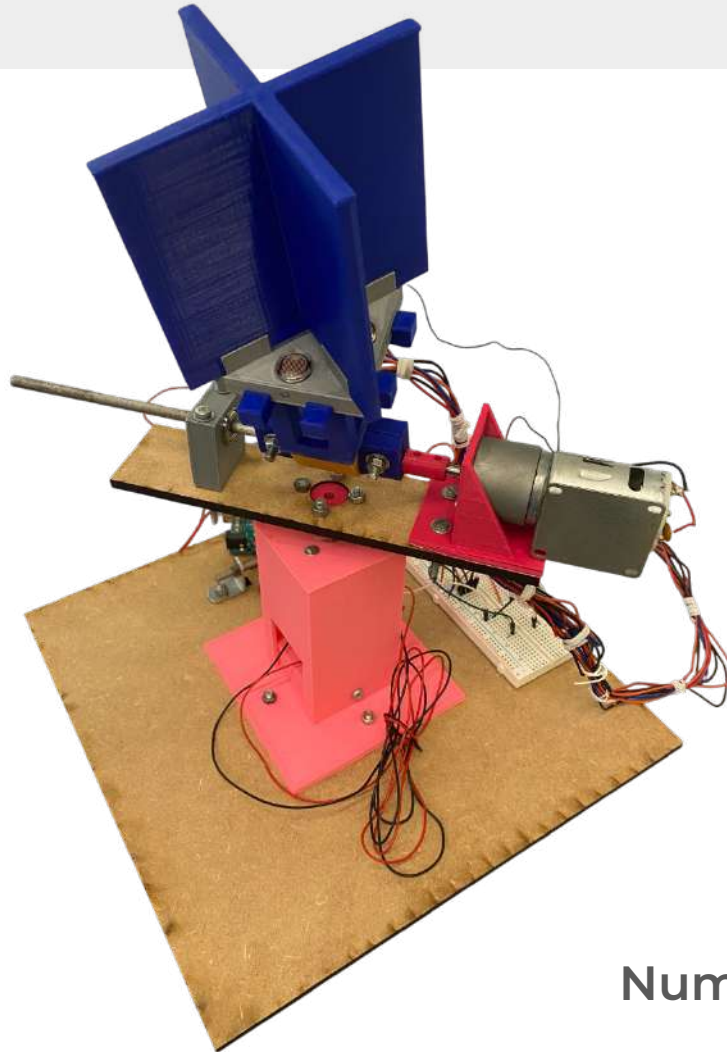


Suiveur solaire pour la récupération d'énergie



Samuel HURTADO
Numéro candidat: 44190

Sommaire

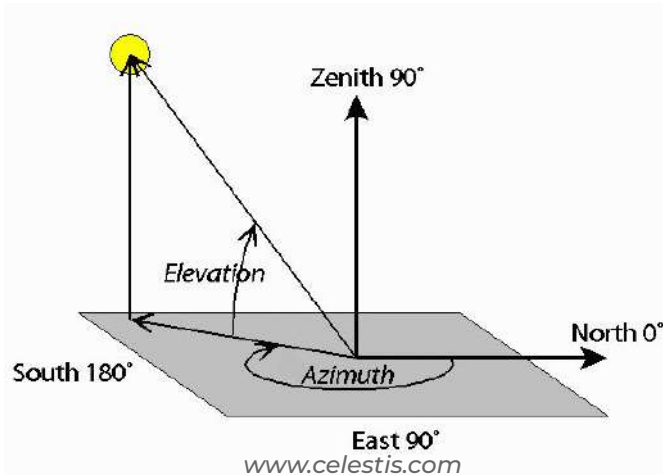
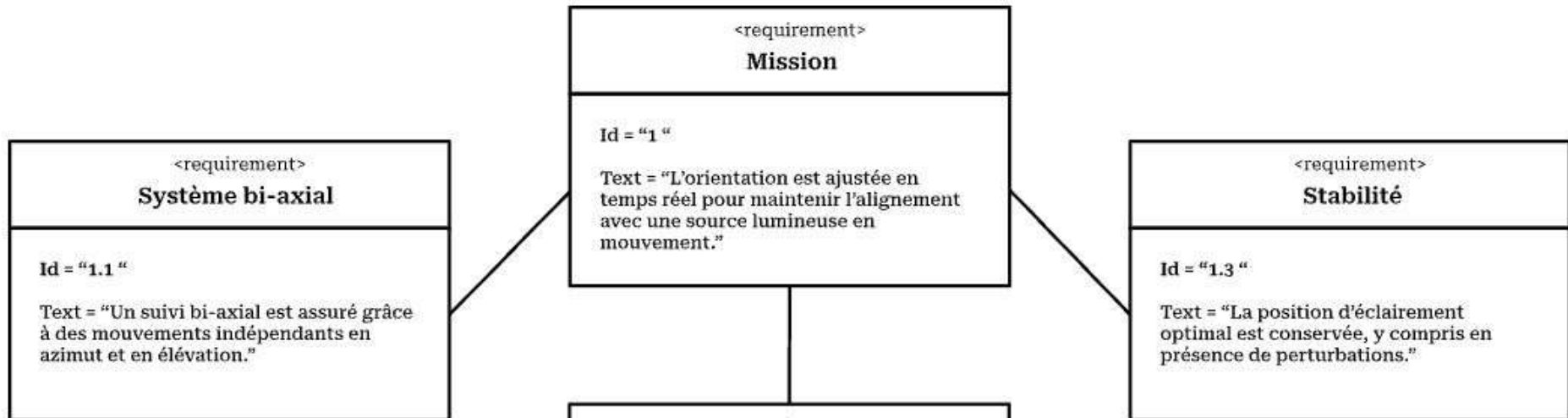
Comment concevoir et optimiser un dispositif modulable qui suit le mouvement du soleil ?

- I - Présentation de la solution envisagée et des contraintes du système
- II - Capteur solaire : conception et exploitation des données
- III - Asservissement dynamique du système
- IV - Conclusion

I - Présentation de la solution envisagée et des contraintes du système

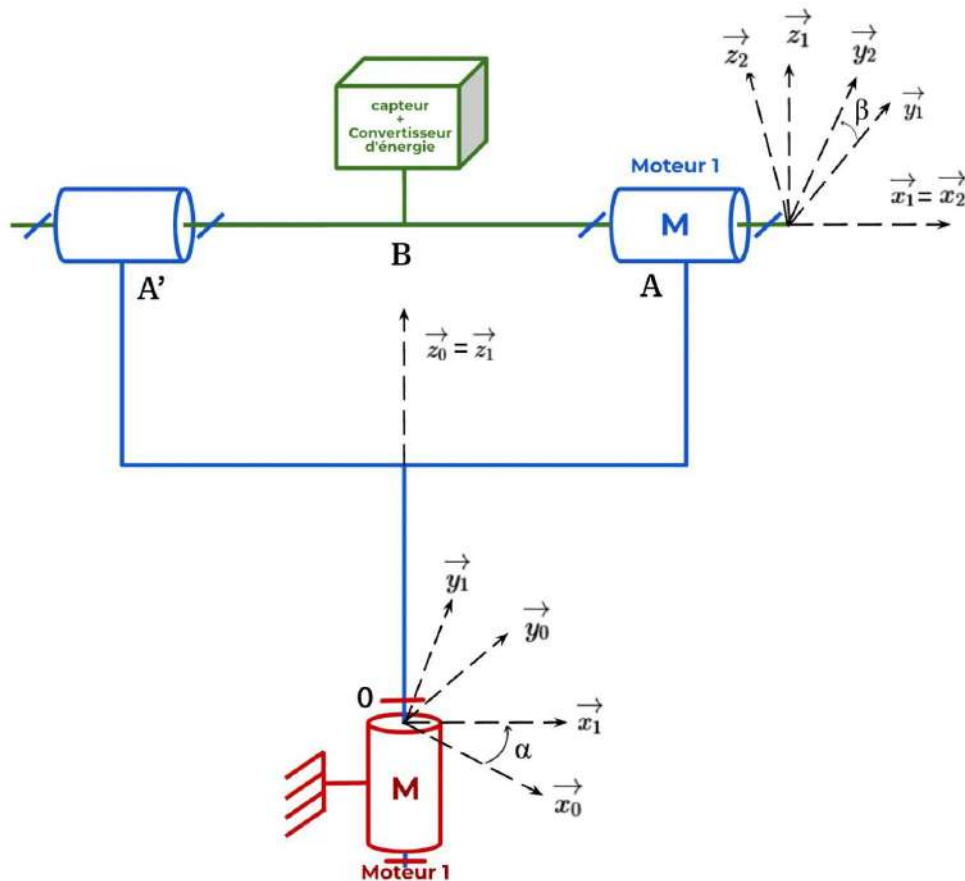
I - Présentation de la solution envisagée et des contraintes du système

Diagramme d'exigence

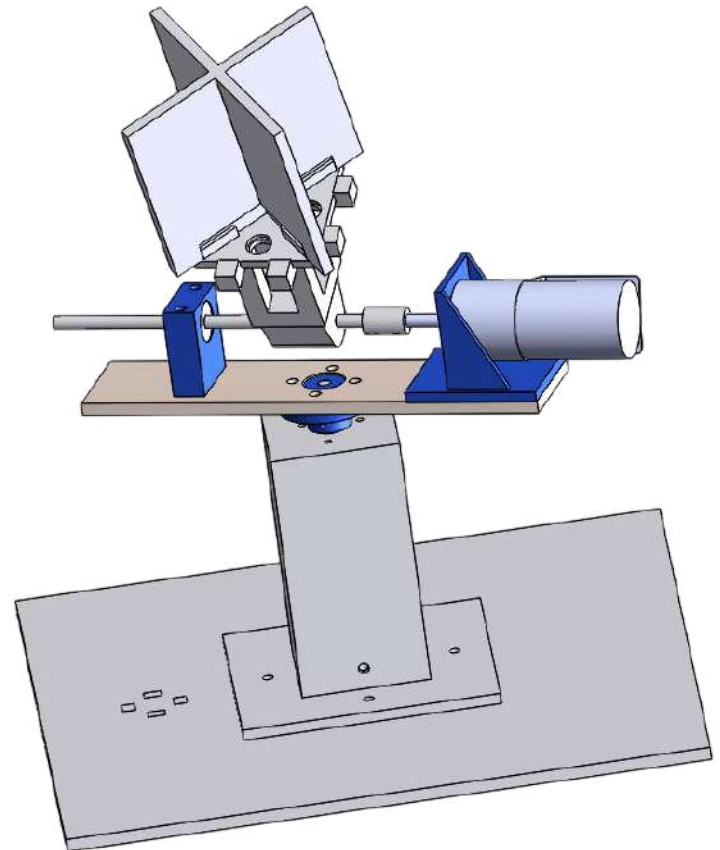


I - Présentation de la solution envisagée et des contraintes du système

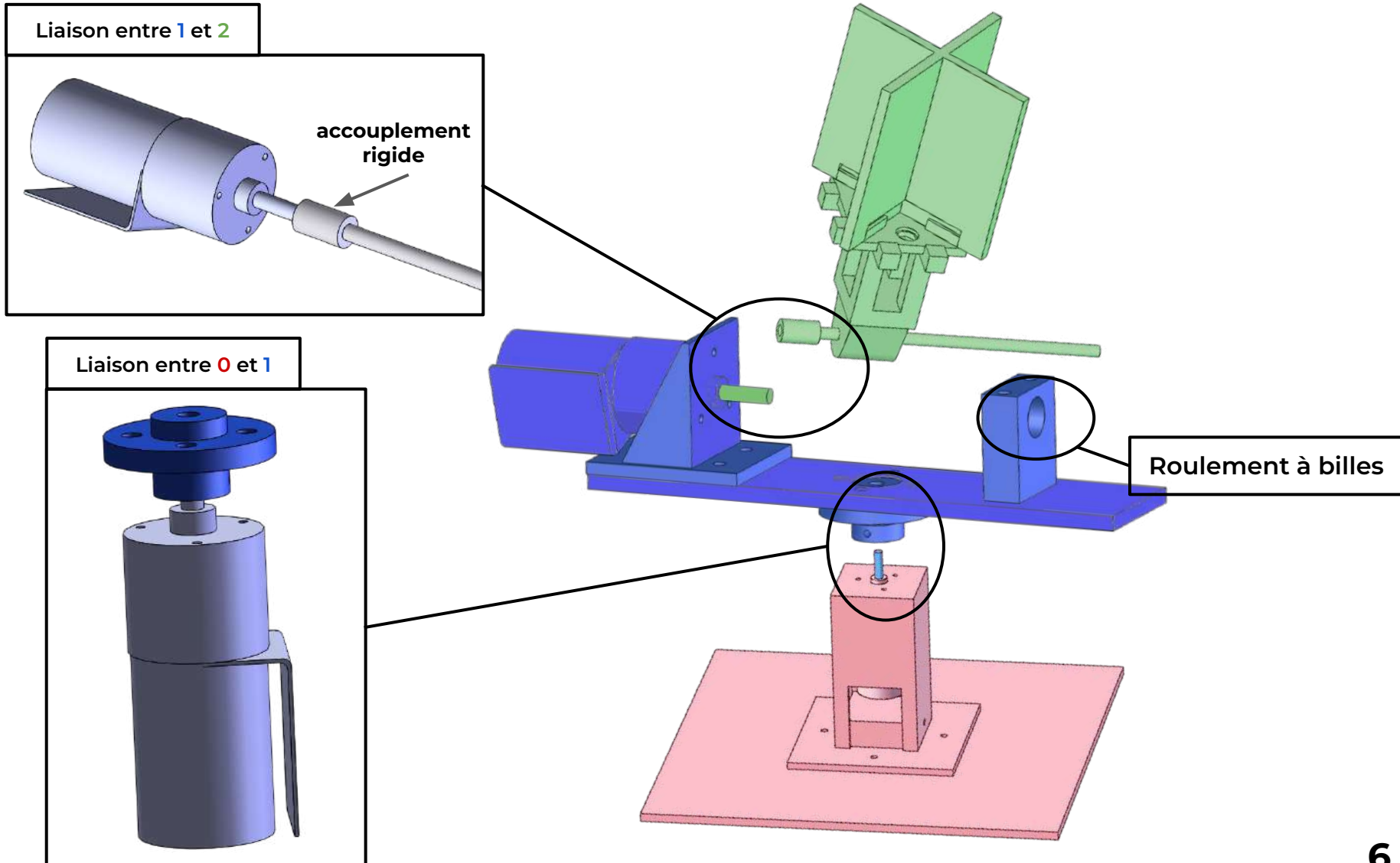
Schéma cinématique en 3D



Modélisation sur SolidWorks



I - Présentation de la solution envisagée et des contraintes du système



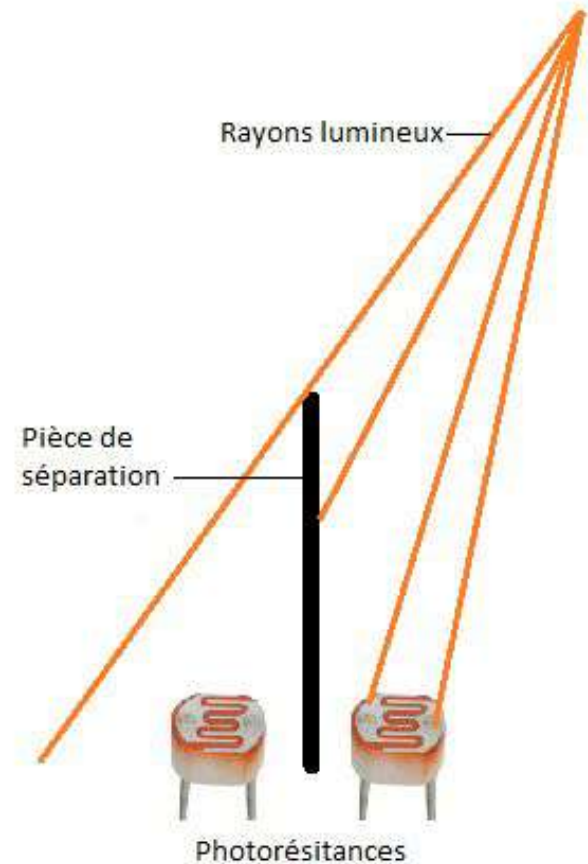
II - Capteur solaire : conception et exploitation des données mesurées

II - Capteur solaire : conception et exploitation des données mesurées

Plastic Package Cds
Photocells NORPS-12



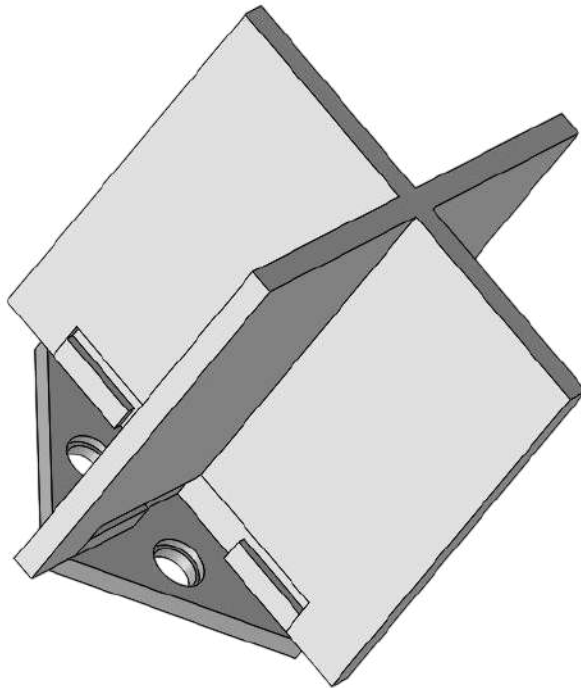
Séparation des photorésistances
pour trouver la position de la source



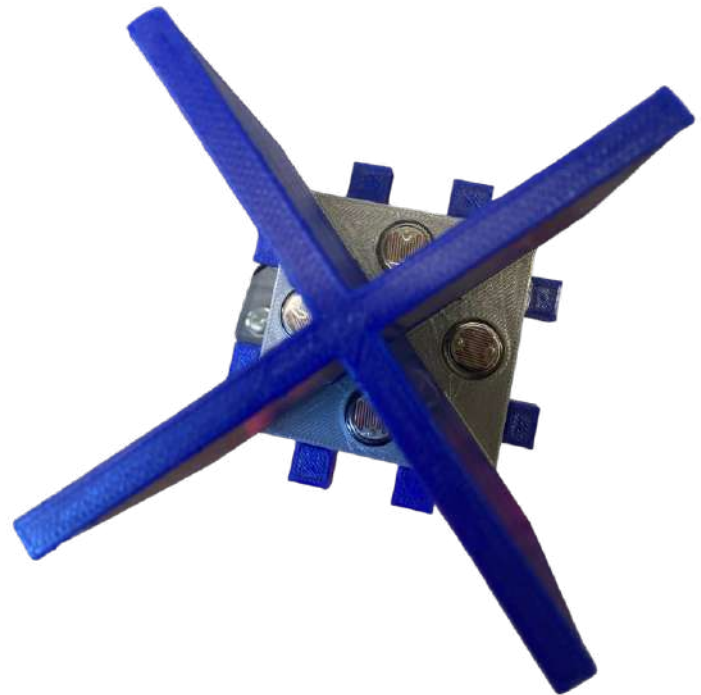
source : Projet GEII- IUT de Mulhouse

II - Capteur solaire : conception et exploitation des données mesurées

Modélisation de la pièce en 3D sur SolidWorks

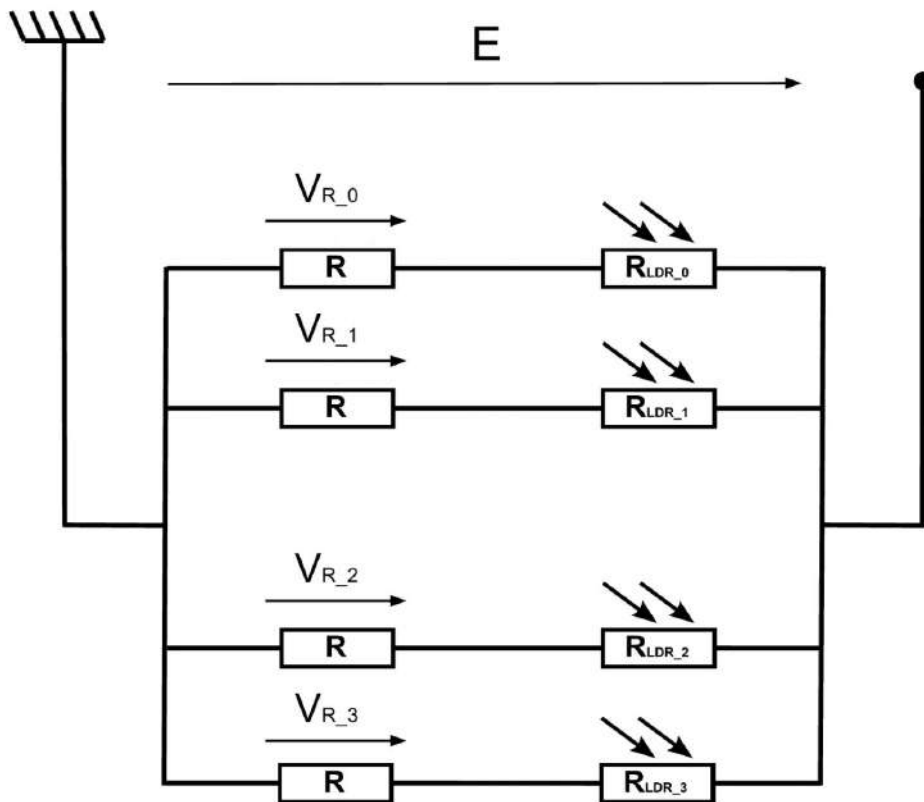


Résultat de l'impression 3D



II - Capteur solaire : conception et exploitation des données mesurées

Schéma électrique



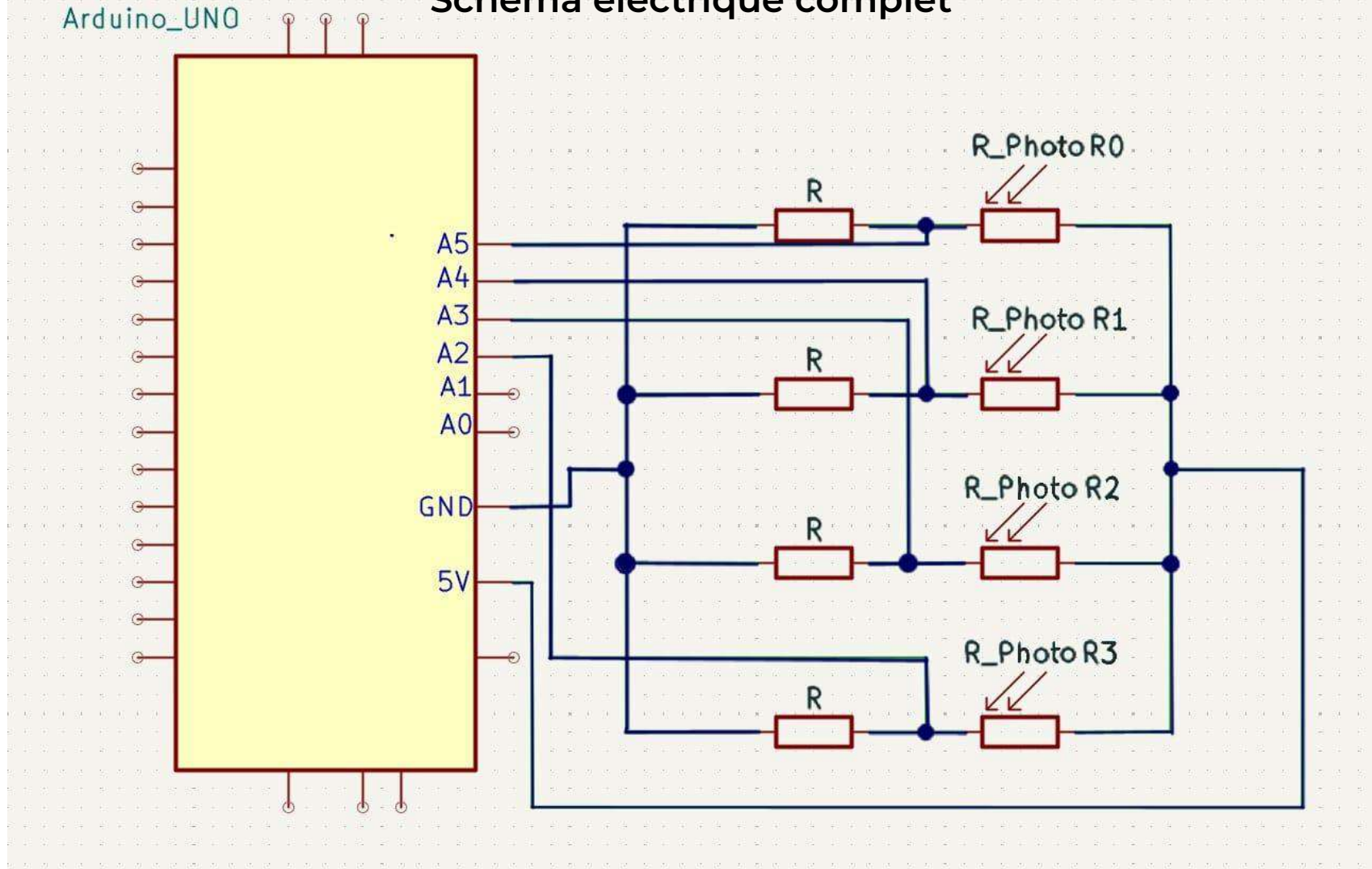
Pont diviseur de tension :

$$\forall i \in [0, 3], V_{R_i} = E \times \frac{R}{R + R_{LDR_i}}$$

$$R_{LDR_i} = R \times \left(\frac{E}{V_{R_i}} - 1 \right)$$

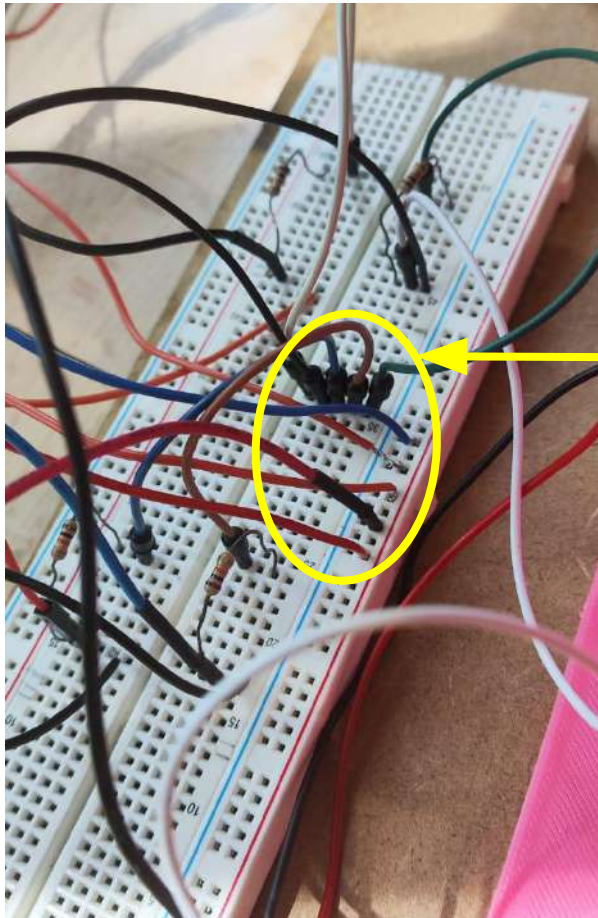
II - Capteur solaire : conception et exploitation des données mesurées

Schéma électrique complet

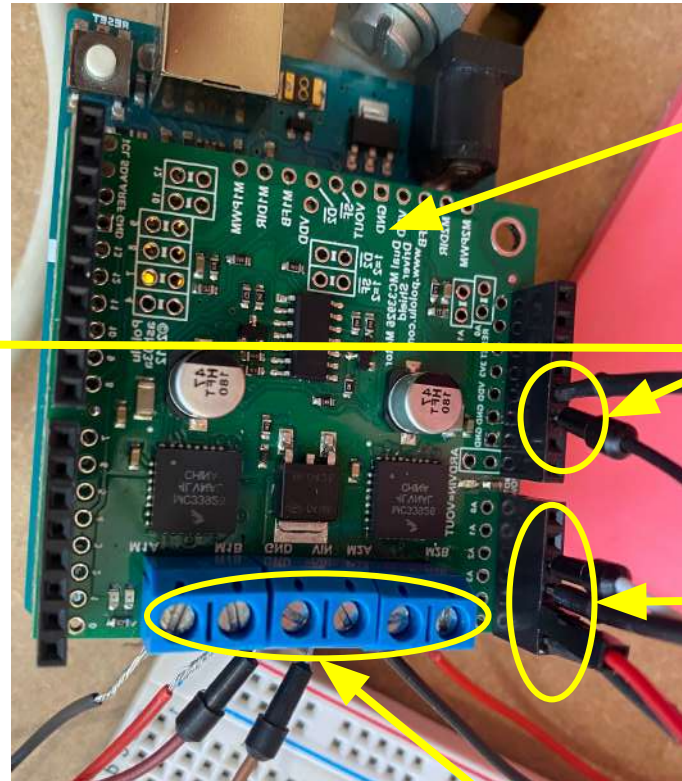


II - Capteur solaire : conception et exploitation des données mesurées

Branchement des LDR



Arduino UNO et Shield



Shield moteur
pour carte
Arduino UNO

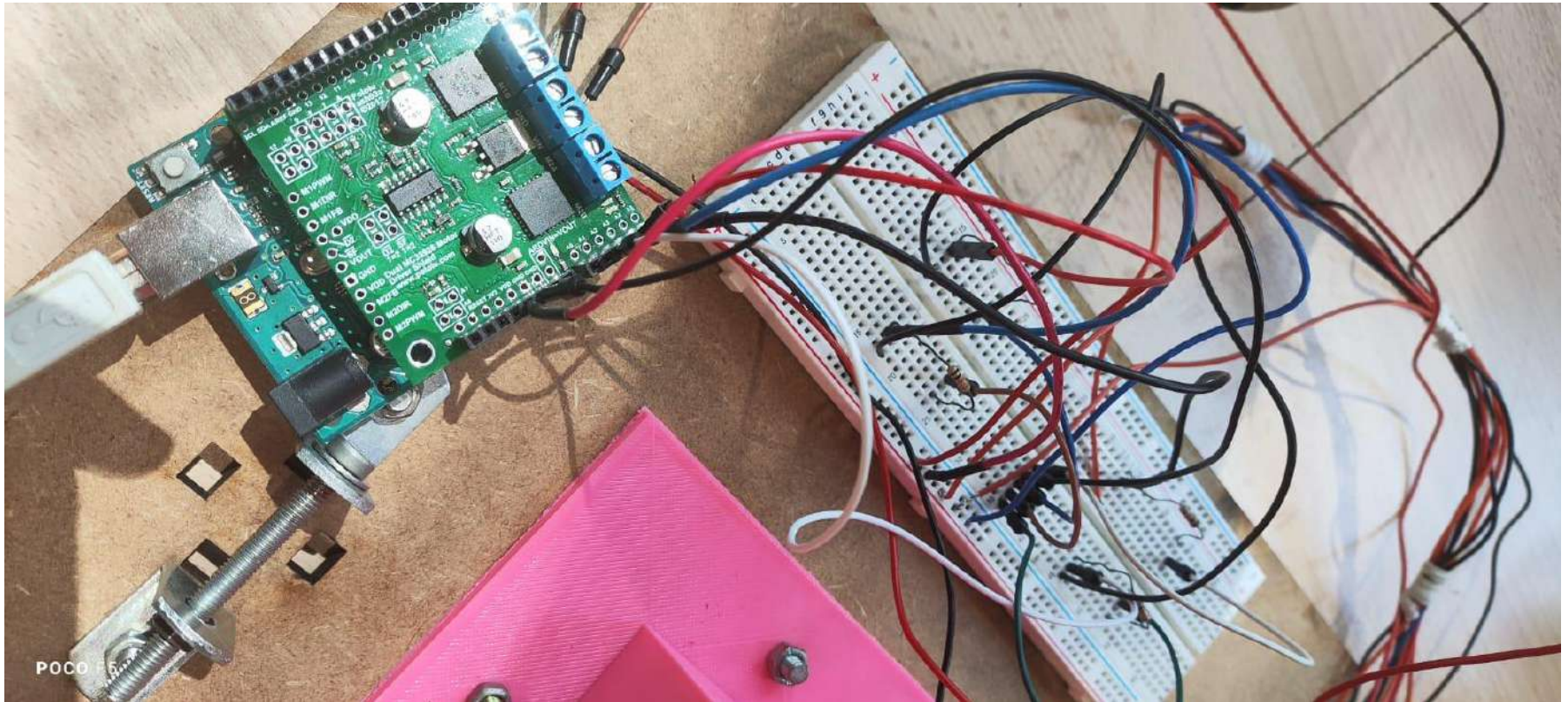
Tension $E = 5V$
et
masse du circuit

Entrées analogiques:
Potentiels entre les
résistances et les
LDR

Connexion des moteurs
et de leur alimentation

II - Capteur solaire : conception et exploitation des données mesurées

Circuit complet



II - Capteur solaire : conception et exploitation des données mesurées

Lecture de valeur de LDR en temps réel

Fort éclairement

Valeur de la LDR 0 : 3098 ohms
Valeur de la LDR 1 : 597 ohms
Valeur de la LDR 2 : 2455 ohms
Valeur de la LDR 3 : 2369 ohms

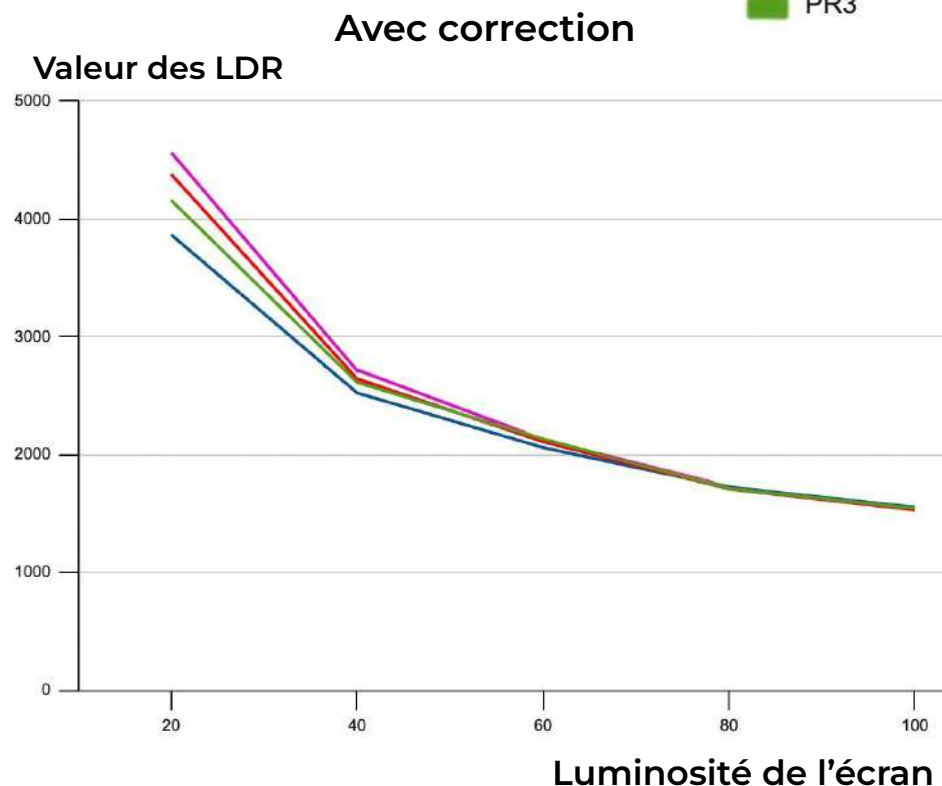
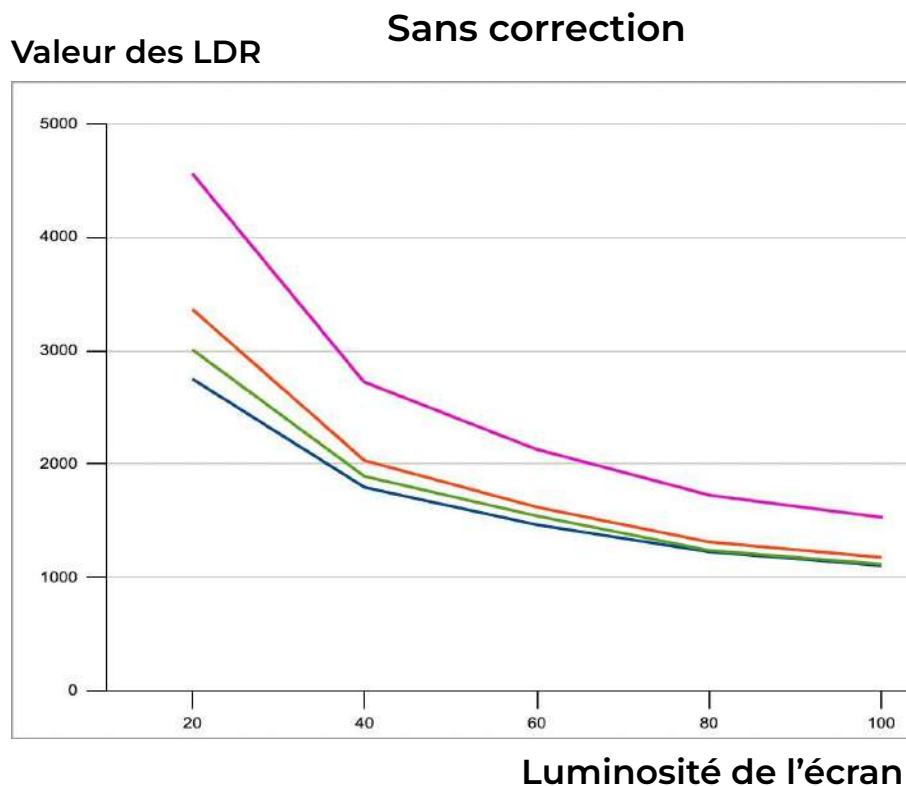
Dans l'obscurité

Valeur de la LDR 0 : 2115 ohms
Valeur de la LDR 1 : 2162 ohms
Valeur de la LDR 2 : 34217 ohms
Valeur de la LDR 3 : 2296 ohms

II - Capteur solaire : conception et exploitation des données mesurées

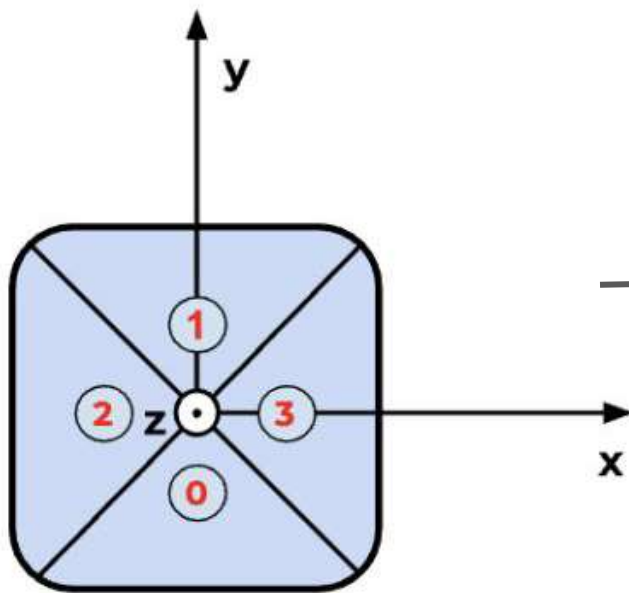
Étalonnage :

- PR0
- PR1
- PR2
- PR3



II - Capteur solaire : conception et exploitation des données mesurées

Capteur solaire



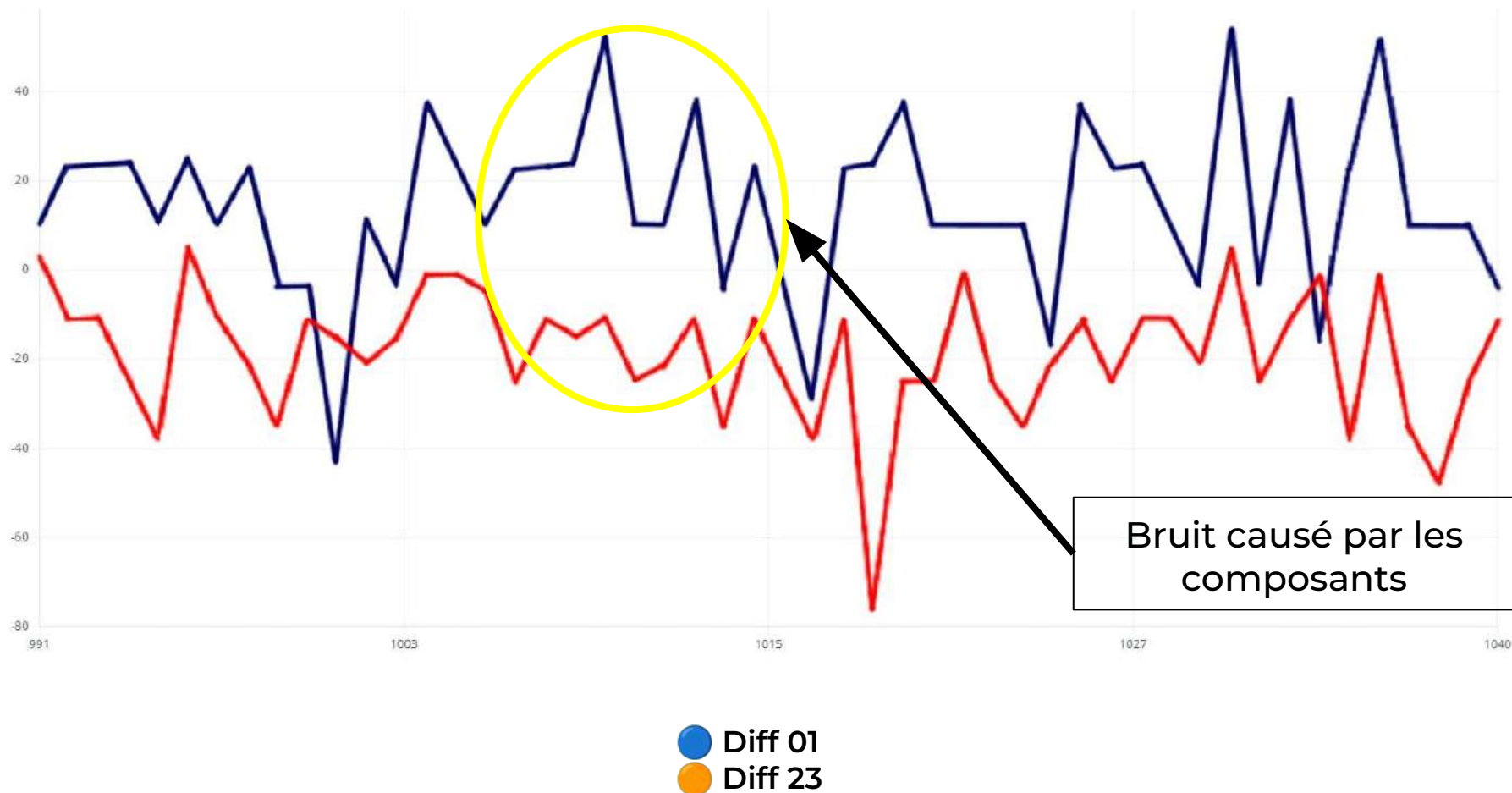
Valeurs utilisées

$$\begin{aligned} Diff01 &= R_{LDR_0} - R_{LDR_1} \\ Diff23 &= R_{LDR_2} - R_{LDR_3} \end{aligned}$$

Objectif : Asservir les moteurs pour que Diff01 et Diff23 tendent vers 0

II - Capteur solaire : conception et exploitation des données

Phénomènes observés sur traceur série de Diff01



II - Capteur solaire : conception et exploitation des données

Fonction de transfert d'un filtre
passe bas du premier ordre :

$$H(\omega) = \frac{S(\omega)}{E(\omega)} = \frac{1}{1 + j\frac{\omega}{\omega_0}}$$

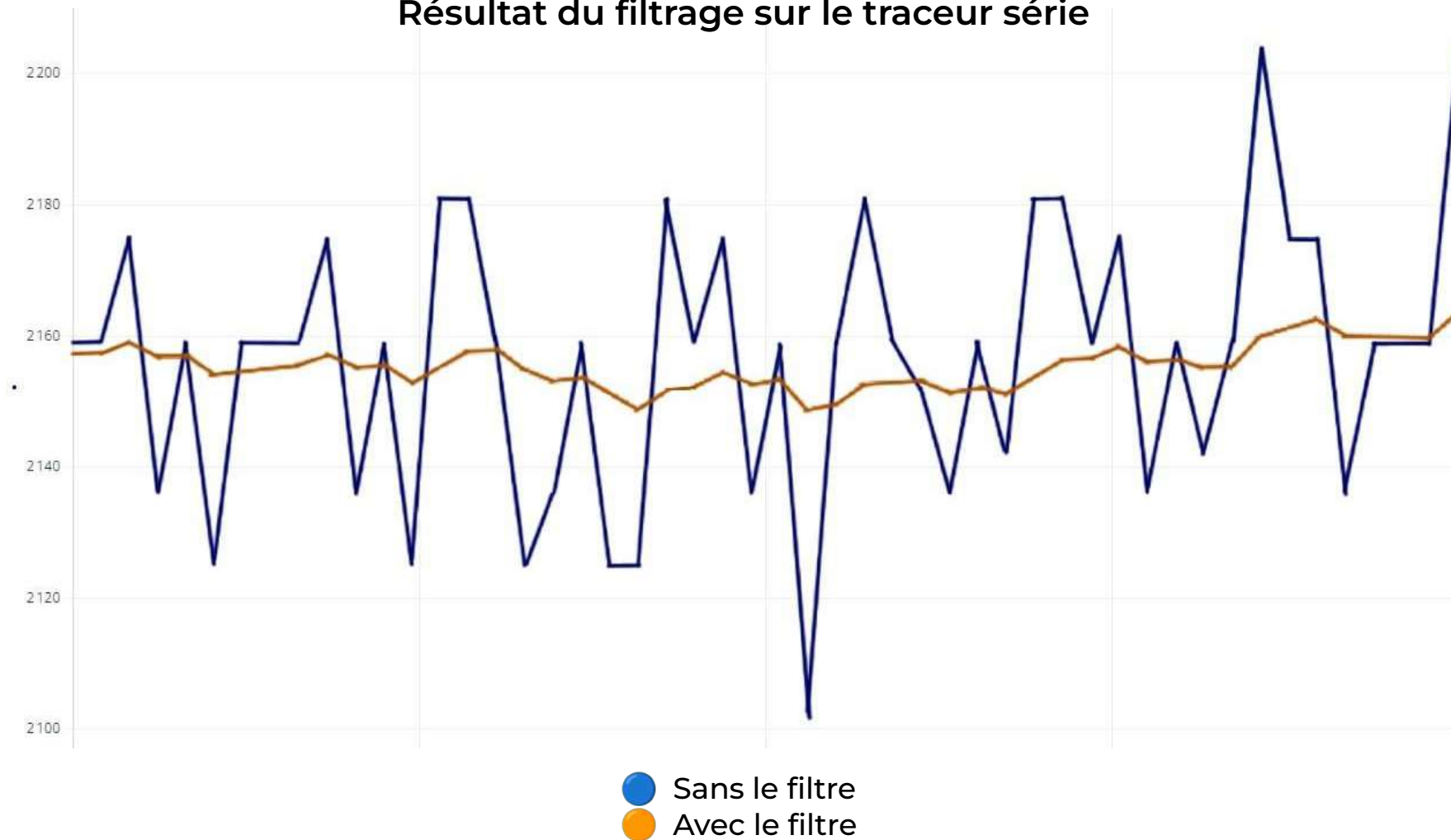
Formule de récurrence du filtre numérique :

$$s[k + 1] = \alpha \times e[k] + s[k] \times (1 - \alpha)$$

$$\text{avec } \alpha = \omega_0 \times Te$$

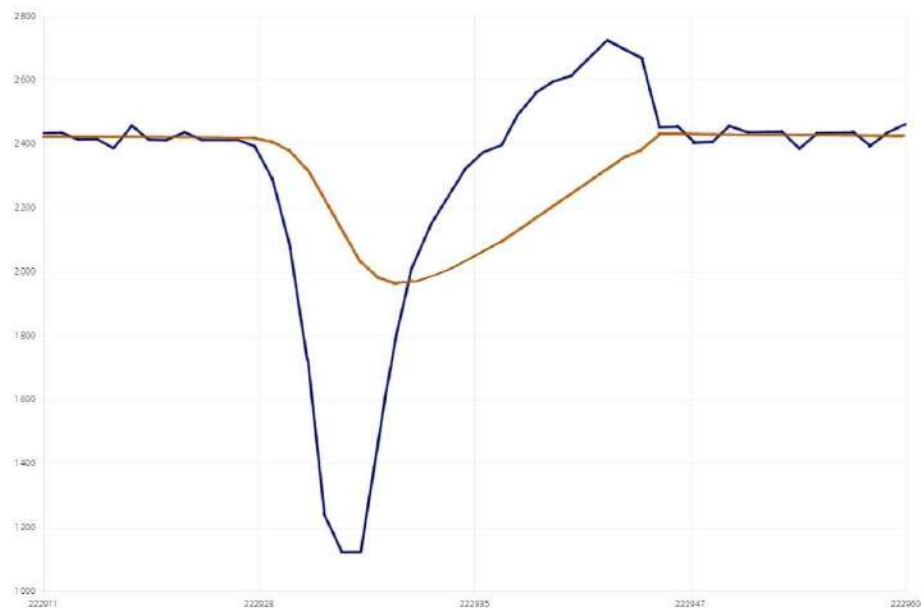
II - Capteur solaire : conception et exploitation des données

Résultat du filtrage sur le traceur série

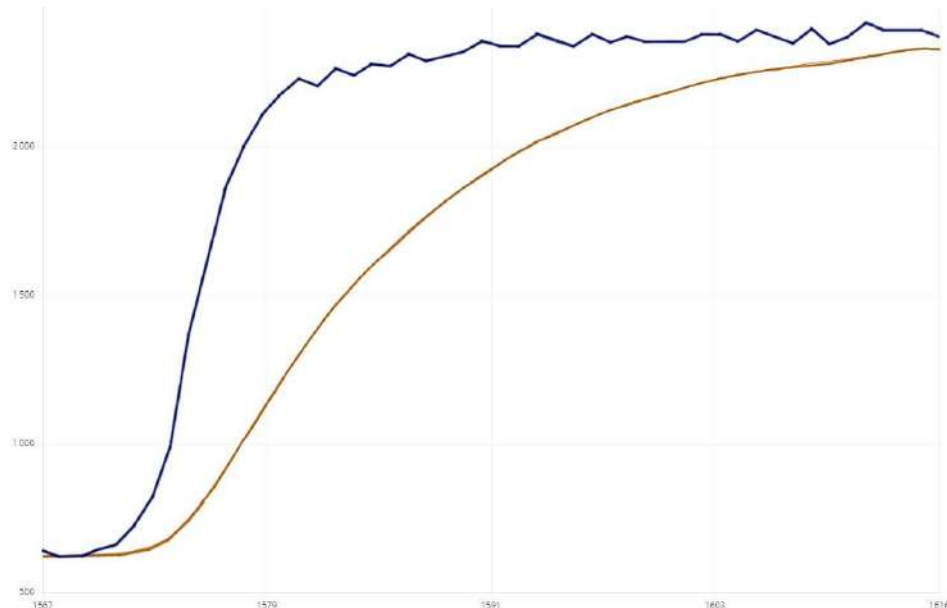


II - Capteur solaire : conception et exploitation des données

Réaction face à une brève perturbation



Changement de position de la source lumineuse

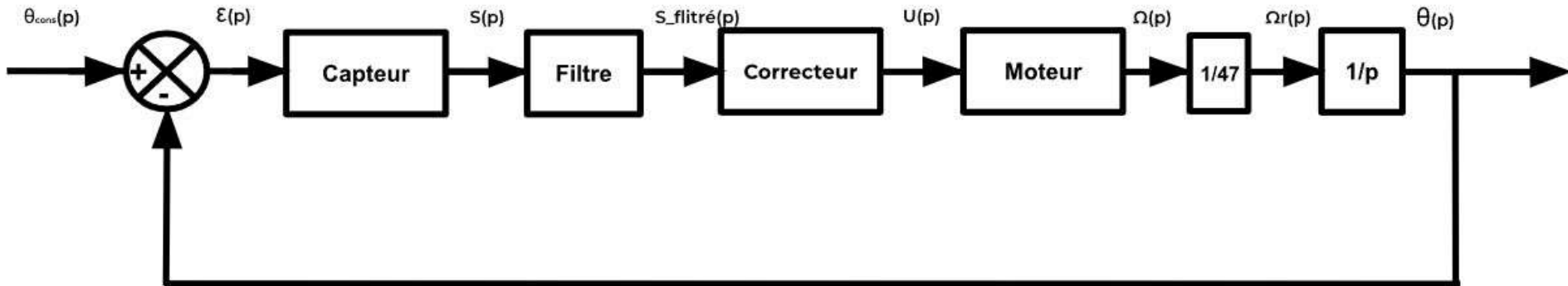


● Diff 01 sans le filtre
● Diff 01 Avec le filtre

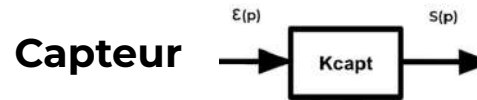
III - Asservissement dynamique du système

III - Asservissement dynamique du système

Schéma bloc fonctionnel

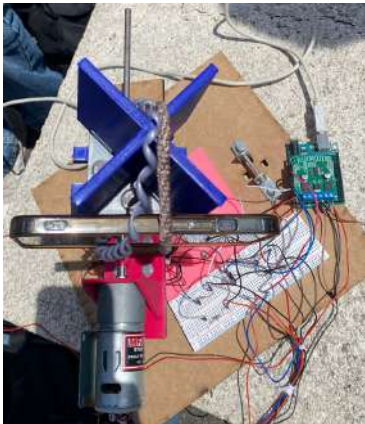


III - Asservissement dynamique du système



$$Diff(t) = K_{capt} \times \theta(t)$$

Capteur en position d'équilibre



Différence ≈ 0

Lecture de l'angle



Application Physic Toolbox

Déplacement de 5°

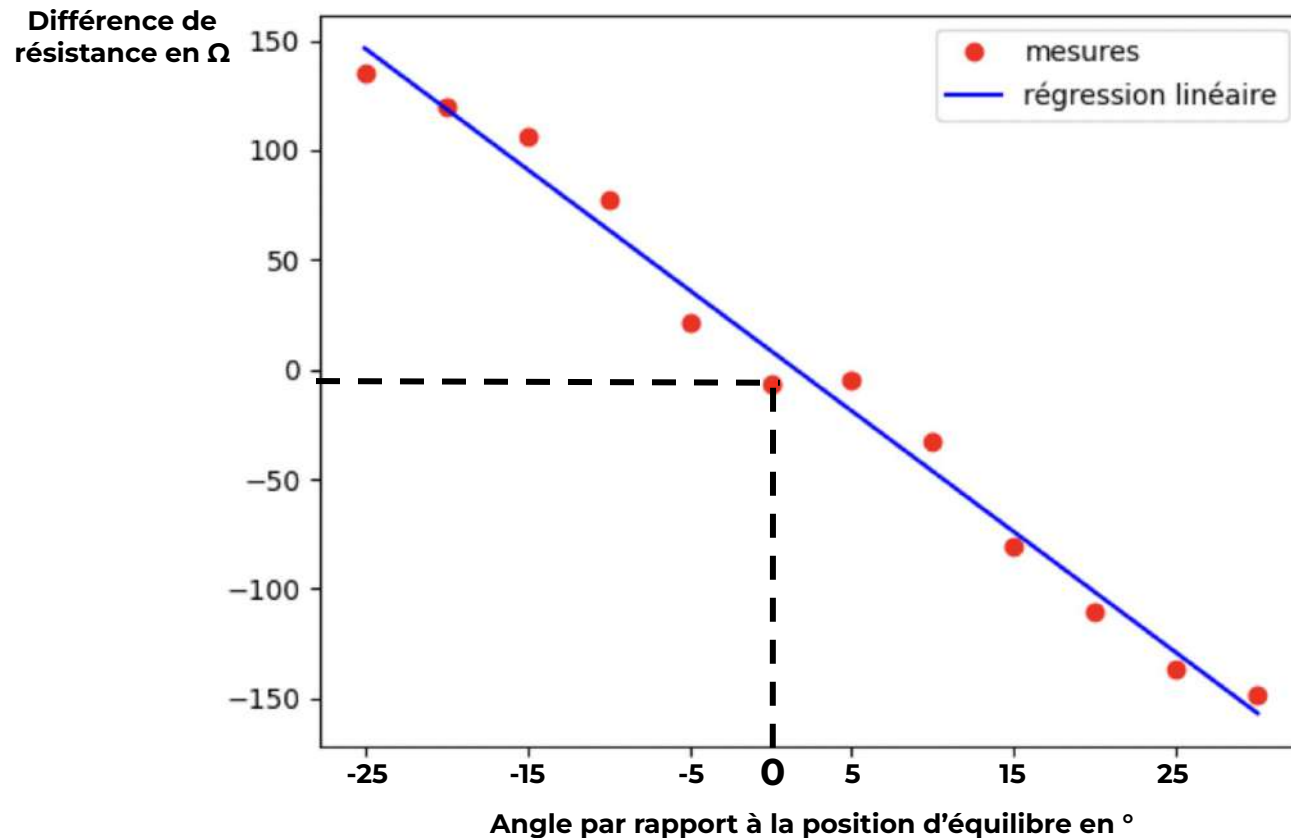


Lecture de la différence sur la console

Diff01 : 2501
Diff23 : 86

III - Asservissement dynamique du système

Linéarisation des mesures



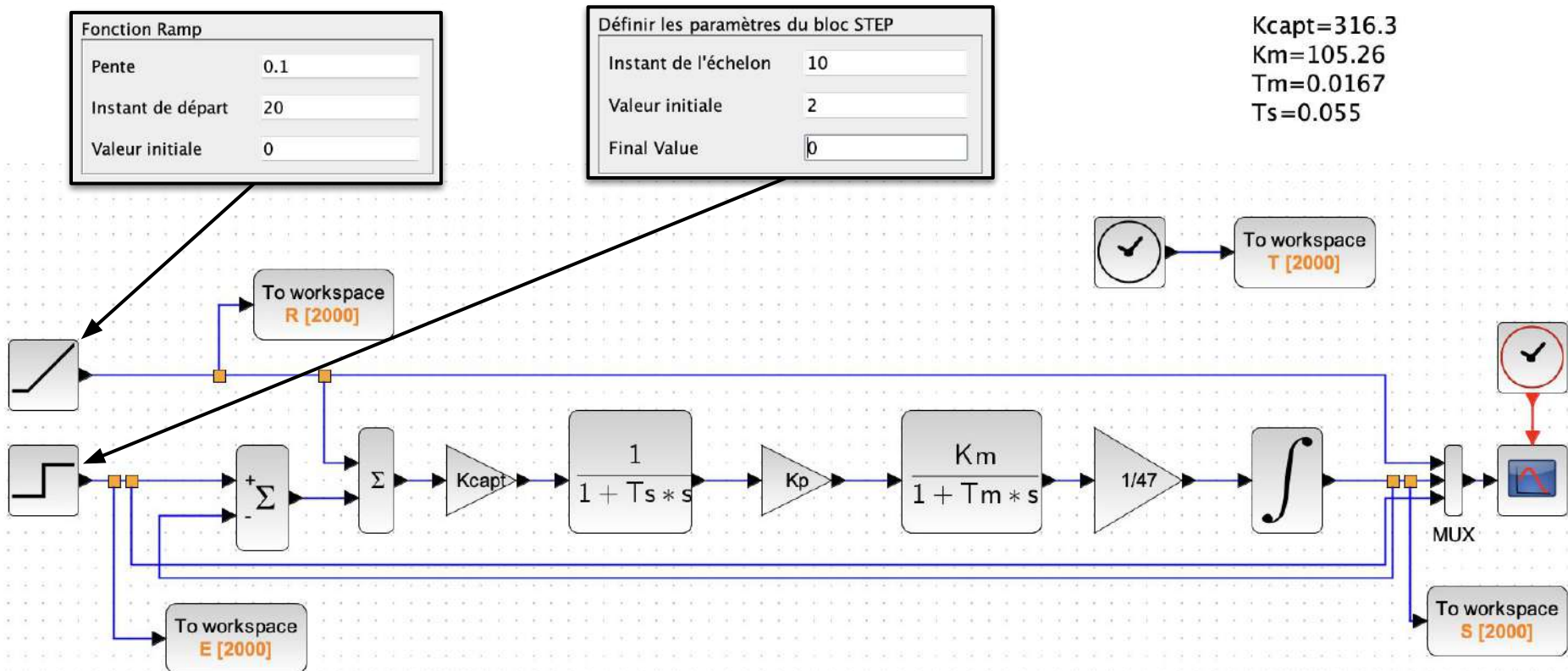
$$K_{\text{capt}} = 5,52 \, \Omega/^{\circ} = \mathbf{316,3 \, \Omega/\text{rad}}$$

III - Asservissement dynamique du système

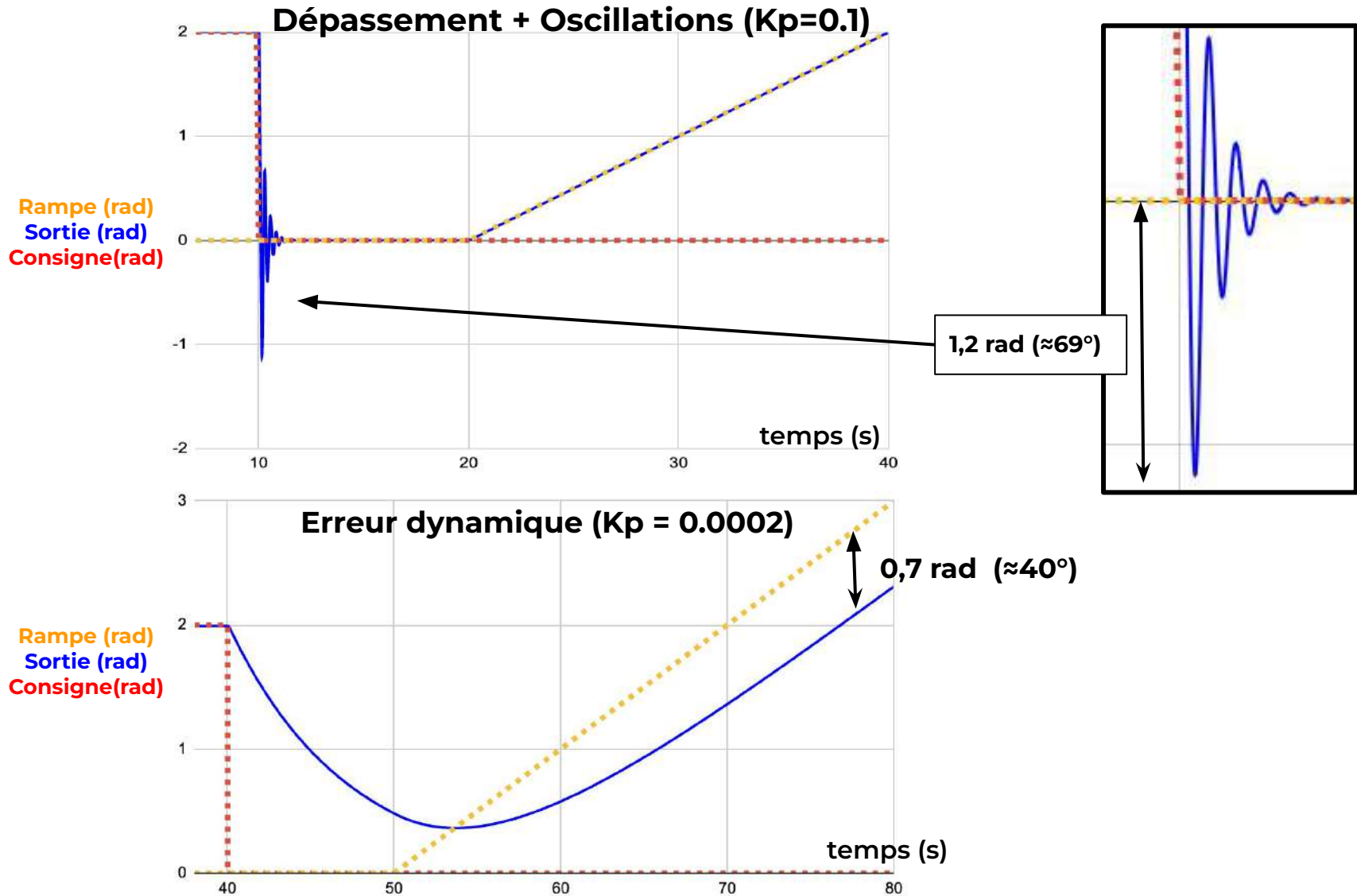
Modélisation de l'asservissement sur Scilab / Xcos :

Contexte

$K_{capt}=316.3$
 $K_m=105.26$
 $T_m=0.0167$
 $T_s=0.055$

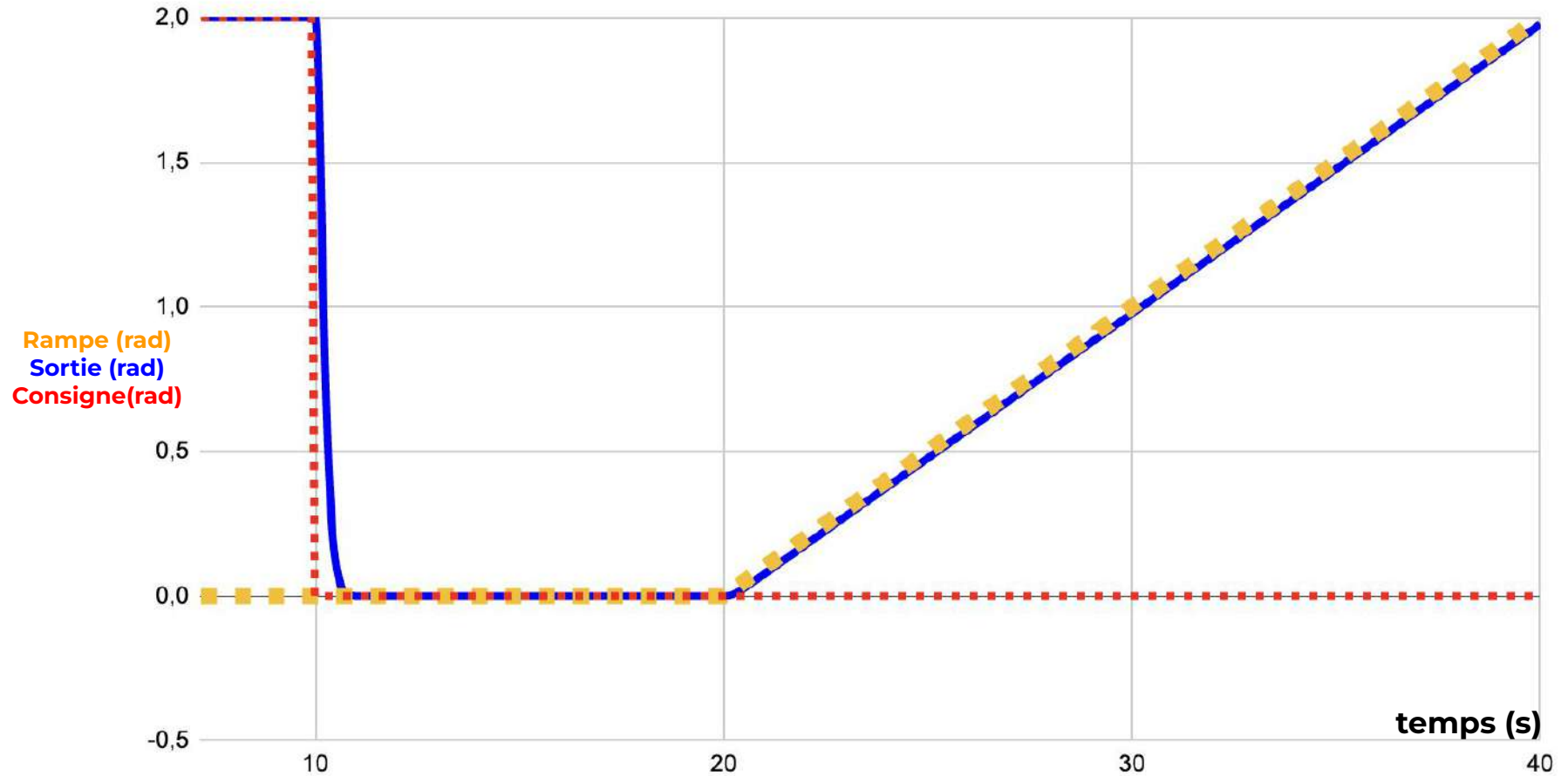


III - Asservissement dynamique du système



III - Asservissement dynamique du système

Gain optimal ($K_p=0.006$)



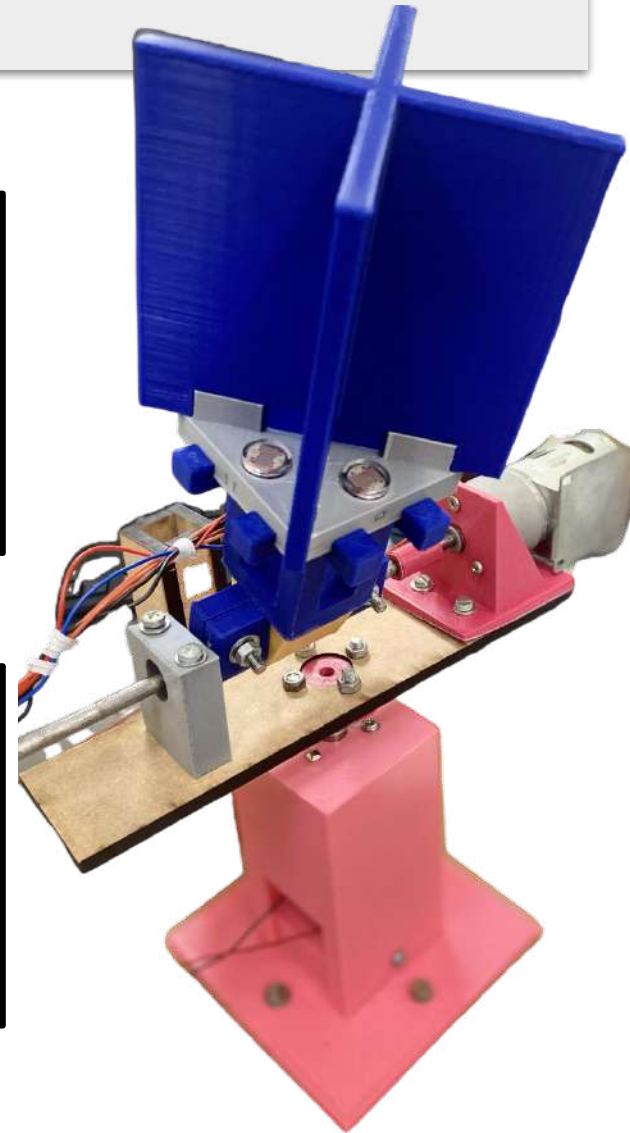
IV - Conclusion

Développement des compétences :

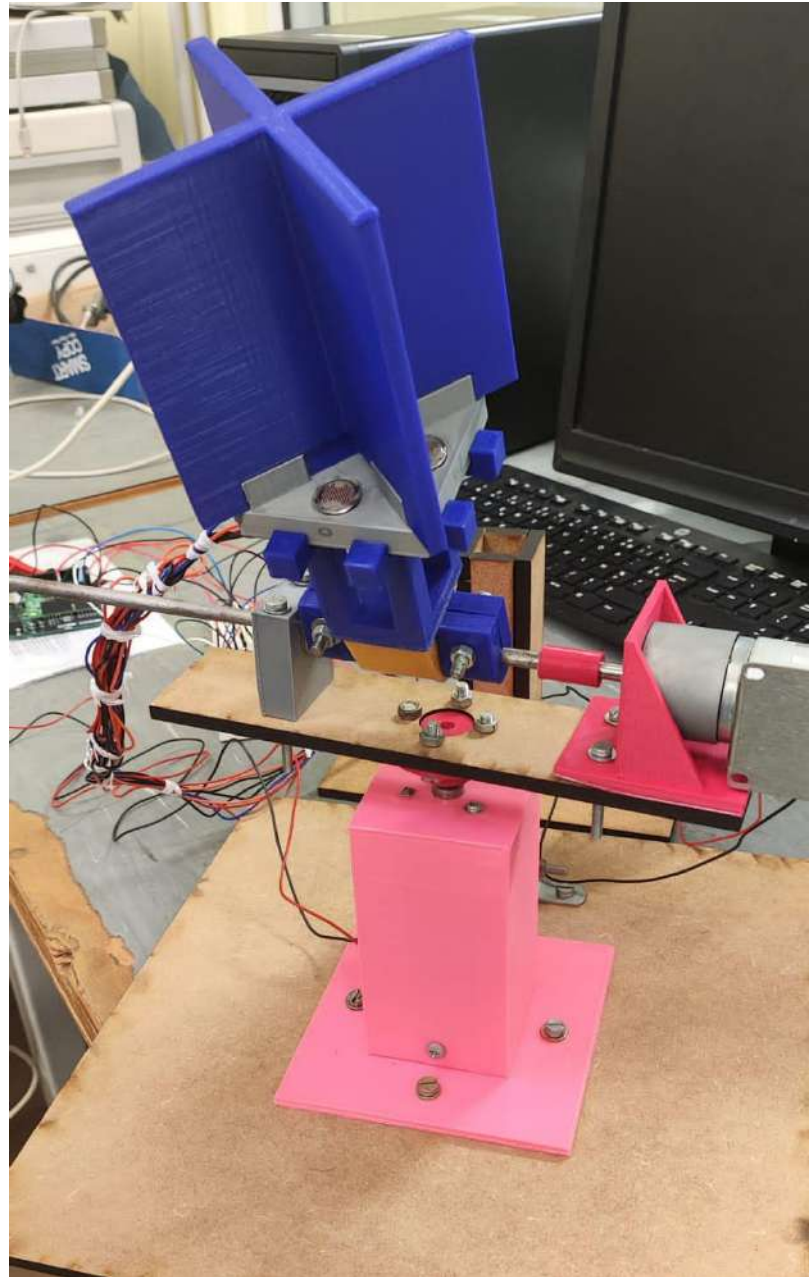
- Électronique et programmation Arduino
- Conception sur SolidWorks
- Simulation sur l'outil Xcos de Scilab
- Approfondissement des notions d'asservissement

Points à améliorer :

- Le bruit des signaux
- Risque de faire plusieurs tours
- Pouvoir supporter plus de poids (miroir, panneau solaire...)

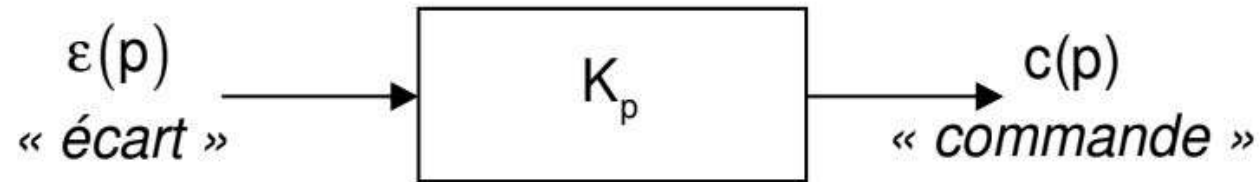


Merci



Annexe 1 :

Correcteur proportionnel



Dans le domaine de Laplace

$$\frac{c(p)}{\varepsilon(p)} = K_p$$

Dans le domaine temporel

$$c(t) = K_p \cdot \varepsilon(t)$$

Annexe 2 :

Formule de récurrence du filtre numérique

Fonction de transfert d'un filtre passe
bas du premier ordre :

$$H_{\underline{}}(\omega) = \frac{\underline{S}(\omega)}{\underline{E}(\omega)} = \frac{1}{1+j\frac{\omega}{\omega_0}}$$

$$\Leftrightarrow (1 + j\frac{\omega}{\omega_0}) \times \underline{S}(\omega) = \underline{E}(\omega)$$

En passant dans le domaine temporel : $\frac{ds}{dt}(t) + \omega_0 \times s(t) = \omega_0 \times e(t)$

Utilisation de la méthode d'Euler : $\frac{ds}{dt}(t) \approx \frac{s_{k+1} - s_k}{Te}$

L'équation différentielle
devient :

$$\frac{s[k+1] - s[k]}{Te} + \omega_0 \times s[k] = \omega_0 \times e[k]$$

On obtient donc : $s[k + 1] = Te \times \omega_0 \times e[k] + s[k] \times (1 - \omega_0 Te)$

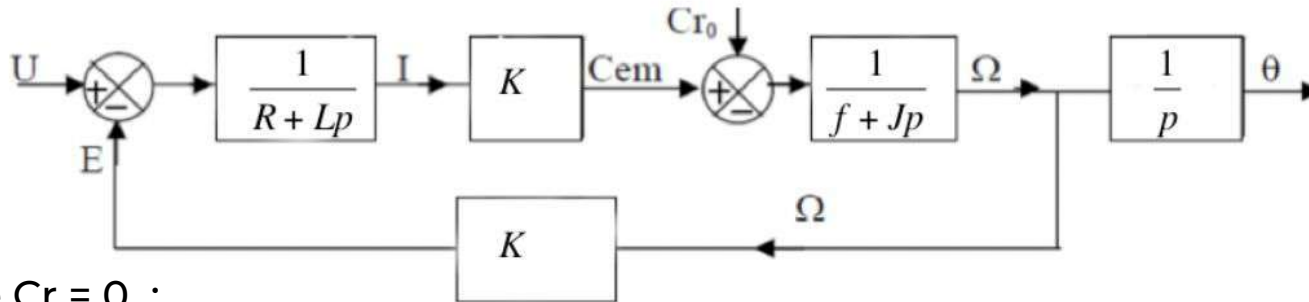
La formule du filtre numérique est donc :

$$s[k + 1] = \alpha \times e[k] + s[k] \times (1 - \alpha)$$

avec $\alpha = \omega_0 \times Te$

Annexe 3 :

Fonction de transfert d'un moteur



Hypothèse de $C_r = 0$:

$$\begin{aligned}
 H(p) &= \frac{\Omega(p)}{U(p)} = \frac{K}{K^2 + (f + Jp)(R + Lp)} = \frac{K/Rf}{\frac{K^2}{Rf} + \left(1 + \frac{J}{f}p\right)\left(1 + \frac{L}{R}p\right)} \\
 &= \frac{K/Rf}{\frac{K^2}{Rf} + 1 + \left(\frac{J}{f} + \frac{L}{R}\right)p + \frac{JL}{fR}p^2} = \frac{K}{K^2 + Rf + Rf\left(\frac{J}{f} + \frac{L}{R}\right)p + JLP^2} \\
 &= \frac{\frac{K}{K^2 + Rf}}{1 + \frac{Rf}{K^2 + Rf}\left(\frac{J}{f} + \frac{L}{R}\right)p + \frac{JL}{K^2 + Rf}p^2}
 \end{aligned}$$

$$K_m = 1/K$$

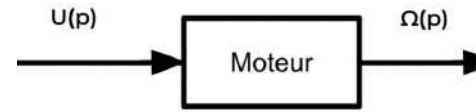
En négligeant l'inductance et les frottements : $H_{\text{moteur}}(p) = \frac{K_m}{1 + \tau_m p}$

$$\tau_m = \frac{R \times J}{K^2}$$

Annexe 4 :

Fonction de transfert d'un moteur

Modélisation du moteur



Stall Current: RE540/1 at 6v = 8.24A

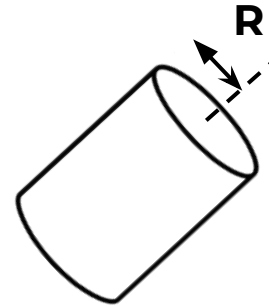
fiche technique du moteur

$$R = \frac{6,00}{8,24} = 0,73 \, \Omega$$

$$J = \frac{MR^2}{2}$$



Rotor du moteur



$$J = \frac{0,057 \cdot 0,005^2}{2}$$

$$J = 1,4 \times 10^{-6} \, \text{kg.m}^2$$

VOLTAGE		SPEED R.P.M.	CURRENT A
OPERATING RANGE	NOMINAL		
4.5 - 15	12v Constant	12048	2.81

$$I = 2,81 \, \text{A}$$

$$K = \frac{U}{\Omega} = \frac{E - R \times I}{\Omega} = 7,88 \times 10^{-3} \, \text{V.s.rad}^{-1}$$

fiche technique du moteur

$$\Omega = 1262 \, \text{rad/s}$$

Annexe 5 :

Fonction de transfert d'un moteur



$$H_{moteur}(p) = \frac{K_m}{1 + \tau_m \times p}$$

$$K_m = 1/K \quad \tau_m = \frac{R \times J_{eq}}{K^2}$$

$$H_{moteur}(p) = \frac{105,26}{1 + 0,0115 \times p}$$

Annexe 6 :

code python de la régression linéaire

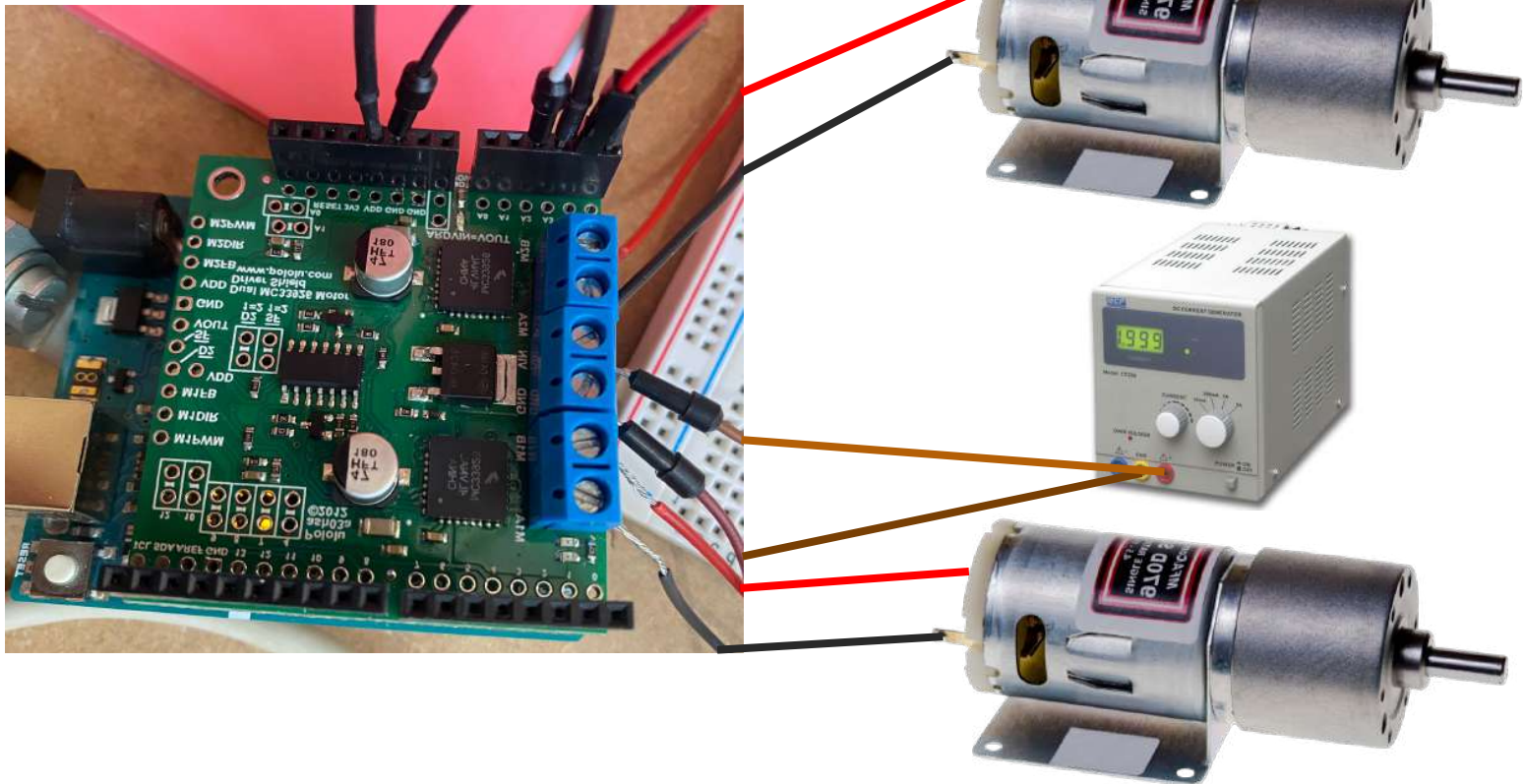
```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([90, 95, 100, 105, 110, 115, 120, 125, 130, 135, 140, 145])
y = np.array([135, 120, 106, 77, 21, -7, -5, -33, -81, -110, -137, -148])
[a, b] = np.polyfit ( x, y, 1) #coefficients du polynôme de degré 1 qui
colle le plus à la courbe y=f(x)

m=plt.plot(x, y,"ro")
ymodel = a * x + b
rl = plt.plot(x, ymodel, 'b-')
plt.legend(m + rl, ['mesures', 'régression linéaire'])
plt.show()
print(a)
```

Annexe 7 :

Branchement des moteurs sur la carte Arduino à l'aide d'un shield



Annexe 8 :

```
1 // *** Code d'asservissement ***
2
3 // Définir les broches pour les moteurs
4 #define D2 4 // Définit la broche 4 comme D2 pour un contrôle ou une indication
5 #define M1DIR 7 // Définit la broche 7 pour la direction du moteur 1
6 #define M2DIR 8 // Définit la broche 8 pour la direction du moteur 2
7 #define M1PWM 9 // Définit la broche 9 pour le PWM (vitesse) du moteur 1
8 #define M2PWM 10 // Définit la broche 10 pour le PWM (vitesse) du moteur 2
9 #define M1FB A0 // Définit la broche A0 pour le retour d'information (feedback) du moteur 1
10 #define M2FB A1 // Définit la broche A1 pour le retour d'information (feedback) du moteur 2
11
12 // Définir les constantes pour les photorésistances
13 #define ResistanceFixe 9980
14 #define brocheEntreePointMilieuPontDiviseurDeTension0 A2
15 #define brocheEntreePointMilieuPontDiviseurDeTension1 A3
16 #define brocheEntreePointMilieuPontDiviseurDeTension2 A4
17 #define brocheEntreePointMilieuPontDiviseurDeTension3 A5
18 #define AlimentationArduino 5
19
20 // Variables pour les photorésistances
21 int valeurEntreeAnalogique0, valeurEntreeAnalogique1, valeurEntreeAnalogique2, valeurEntreeAnalogique3;
22 float tensionEntreeAnalogique0, tensionEntreeAnalogique1, tensionEntreeAnalogique2, tensionEntreeAnalogique3;
23 long valeurOhmiqueDeLaLDR0, valeurOhmiqueDeLaLDR1, valeurOhmiqueDeLaLDR2, valeurOhmiqueDeLaLDR3;
24 long valeurOhmLDR0AvecCorrection, valeurOhmLDR1AvecCorrection, valeurOhmLDR3AvecCorrection;
25
26 // Constantes pour l'asservissement
27 float Kp = 0.006;
28 float commandeActu;
29 float commandePasse;
30 float ecartPasse;
31 // Constantes pour le filtre
32 const float alpha01 = 1.0; // coefficient de filtrage
33 const float alpha23 = 1.0; // coefficient de filtrage
34 float diff01_filtre = 0.0;
35 float diff23_filtre = 0.0;
```


Annexe 9 :

```
37 void lirePhotocapteurs() {
38
39     // Lecture de l'entrée analogique (cela retourne une valeur sur 10 bits, comprise entre 0 et 1023)
40     valeurEntreeAnalogique0 = analogRead(brocheEntreePointMilieuPontDiviseurDeTension0);
41     valeurEntreeAnalogique1 = analogRead(brocheEntreePointMilieuPontDiviseurDeTension1);
42     valeurEntreeAnalogique2 = analogRead(brocheEntreePointMilieuPontDiviseurDeTension2);
43     valeurEntreeAnalogique3 = analogRead(brocheEntreePointMilieuPontDiviseurDeTension3);
44
45     // Détermination de la tension présente sur l'entrée analogique
46     tensionEntreeAnalogique0 = float(AlimentationArduino * valeurEntreeAnalogique0) / 1023;
47     tensionEntreeAnalogique1 = float(AlimentationArduino * valeurEntreeAnalogique1) / 1023;
48     tensionEntreeAnalogique2 = float(AlimentationArduino * valeurEntreeAnalogique2) / 1023;
49     tensionEntreeAnalogique3 = float(AlimentationArduino * valeurEntreeAnalogique3) / 1023;
50
51     // Détermination de la valeur de la LDR, en fonction de la valeur de la résistance fixe et de la tension
52     // →  $LDR = R \cdot (V_{cc} - V_s) / V_s$ 
53     // → d'où  $valLDR = R \cdot (tensionAlimArduino - tensionEntreeAnalogique) / tensionEntreeAnalogique$ 
54     valeurOhmiqueDeLaLDR0 = ResistanceFixe * (AlimentationArduino / tensionEntreeAnalogique0 - 1);
55     valeurOhmiqueDeLaLDR1 = ResistanceFixe * (AlimentationArduino / tensionEntreeAnalogique1 - 1);
56     valeurOhmiqueDeLaLDR2 = ResistanceFixe * (AlimentationArduino / tensionEntreeAnalogique2 - 1);
57     valeurOhmiqueDeLaLDR3 = ResistanceFixe * (AlimentationArduino / tensionEntreeAnalogique3 - 1);
58
59     valeurOhmLDR0AvecCorrection = valeurOhmiqueDeLaLDR0 / 0.7111;
60     valeurOhmLDR1AvecCorrection = valeurOhmiqueDeLaLDR1 / 0.7682;
61     valeurOhmLDR3AvecCorrection = valeurOhmiqueDeLaLDR3 / 0.7234;
62 }
```

Annexe 10 :

```
64 void setup() {
65     // Initialisation des moteurs
66     pinMode(D2, OUTPUT);    // Configure D2 comme sortie
67     pinMode(M1DIR, OUTPUT); // Configure M1DIR comme sortie
68     pinMode(M1PWM, OUTPUT); // Configure M1PWM comme sortie
69     pinMode(M2DIR, OUTPUT); // Configuration commentée pour le moteur 2
70     pinMode(M2PWM, OUTPUT); // Configuration commentée pour le moteur 2
71
72     // Initialisation des registres pour régler la fréquence du PWM :
73     TCCR1A = (0 << WGM11) | (0 << WGM10) | (0 << COM1A0) | (1 << COM1A1) | (0 << COM1B0) | (1 << COM1B1);
74     TCCR1B = (1 << ICNC1) | (1 << ICES1) | (1 << WGM13) | (0 << WGM12) | (0 << CS12) | (0 << CS11) | (1 << CS10);
75     TCNT1 = 0;                // Réinitialise le compteur du Timer 1
76     ICR1 = 500;              // Définit la valeur de comparaison pour le PWM
77     digitalWrite(D2, HIGH); // Met D2 à l'état HIGH (vérification ou activation)
78
79     moteur1(0); // Arrête le moteur 1 au démarrage
80     moteur2(0); // Arrête le moteur 2 au démarrage
81
82     // Initialisation de la communication série
83     Serial.begin(9600);
84     Serial.println("Debut de l'asservissement en position");
85 }
```

Annexe 11 :

```
//Correcteur
int correcteurP(float ecartActu, float valMax)
{
    float commande = Kp * ecartActu;
    if (commande >= valMax) {
        commande = valMax;
    }
    else if (commande <= -valMax) {
        commande = -valMax;
    }
    ecartPasse = ecartActu;
    commandePasse = commandeActu;
    return int(commande);
}
```


Annexe 12 :

```
101 void loop() {
102     // Lecture des valeurs des photorésistances
103     lirePhotocapteurs();
104
105     // Calcul des écarts entre les photorésistances
106     int diff01 = valeurOhmLDR0AvecCorrection - valeurOhmLDR1AvecCorrection;
107     int diff23 = valeurOhmiqueDeLaLDR2 - valeurOhmLDR3AvecCorrection;
108
109     //filtre
110     diff01_filtre = (alpha01 * diff01) + ((1.0 - alpha01) * diff01_filtre);
111     diff23_filtre = (alpha23 * diff23) + ((1.0 - alpha23) * diff23_filtre);
112
113     // Calcul des commandes moteurs
114     int commandeMoteur1 = correcteurPI(diff01_filtre, 500); // Commande pour Moteur 1
115     int commandeMoteur2 = correcteurPI(-diff23_filtre, 500); // Commande pour Moteur 2
116
117     // Appliquer les commandes aux moteurs
118     moteur1(commandeMoteur1);
119     moteur2(commandeMoteur2);
120 }
```

Annexe 13 :

```
122 void moteur1(int commande) {
123     // Définir la valeur PWM maximale pour 10 tours/min
124     const int maxPWM = 80; // Exemple : 127 correspond à 10 tours/min
125
126     // Limiter la commande à la valeur maximale
127     if (commande > maxPWM) {
128         commande = maxPWM;
129     } else if (commande < -maxPWM) {
130         commande = -maxPWM;
131     }
132
133     // Le moteur tourne que dans un sens
134     if (commande >= 0) {
135         digitalWrite(M1DIR, HIGH); // Définit la direction du moteur 1 vers l'avant
136         OCR1A = commande;          // Ajuste la valeur PWM pour le moteur 1
137     } else {
138         digitalWrite(M1DIR, LOW);  // Définit la direction du moteur 1 vers l'arrière
139         OCR1A = abs(commande);      // Utilise la valeur absolue pour le PWM
140     }
141 }
```

Annexe 14 :

```
143 void moteur2(int commande) {
144     // Définir la valeur PWM maximale pour 10 tours/min
145     const int maxPWM = 80; // Exemple : 127 correspond à 10 tours/min
146
147     // Limiter la commande à la valeur maximale
148     if (commande > maxPWM) {
149         commande = maxPWM;
150     } else if (commande < -maxPWM) {
151         commande = -maxPWM;
152     }
153
154     if (commande >= 0) {
155         digitalWrite(M2DIR, HIGH); // Définit la direction du moteur 2 vers l'avant
156         OCR1B = commande;          // Ajuste la valeur PWM pour le moteur 2
157     } else {
158         digitalWrite(M2DIR, LOW);  // Définit la direction du moteur 2 vers l'arrière
159         OCR1B = abs(commande);      // Utilise la valeur absolue pour le PWM
160     }
161 }
```

Annexe 15 :

Utilisations possibles :

Panneaux solaires utilisant un tracker



Entreprise GIROSUN

Télescopes solaires



DKIST, Hawaii

Réflecteurs paraboliques



Parabole-Stirling d'Odeillo